

RETO 01-RA4

Mini entorno de trabajo con Jupyter Notebook y Apache Airflow

Big Data Aplicado – RA4

Antonio Ferrer Martínez

Curso 2025–2026

Índice

Ejercicio 1: Instalación y configuración del entorno.....	3
1.1 docker-compose.yml – Definición de los servicios.....	3
Servicio: postgres	3
Servicio: airflow-init	4
Servicio: airflow-webserver	5
Servicio: airflow-scheduler	5
Servicio: jupyter	6
Red y volúmenes	7
1.2 airflow.Dockerfile – Línea por línea.....	7
1.3 jupyter.Dockerfile – Línea por línea.....	8
1.4 Warning extra solucionado en el notebook.....	10
1.5 Puesta en marcha y verificación.....	10
1.6 Enlace para descargar el entorno.....	13
Ejercicio 2: Jupyter Notebook como cliente de Airflow.....	14
2.1 El DAG test1 (del vídeo de Albert Coronado).....	14
2.2 API REST de Airflow desde Jupyter.....	14
2.3 Error controlado con AirflowFailException.....	16
2.4 CLI de Airflow desde Jupyter.....	17
2.5 Ventajas de usar Jupyter como cliente de Airflow.....	18
Ejercicio 3: Integración de Airflow con Jupyter mediante Papermill.....	19
3.1 Notebook parametrizable.....	19
3.2 Prueba local desde Jupyter.....	19
3.3 DAG dag_papermill.....	19
3.4 Ejecución desde Jupyter vía API REST.....	20
3.5 Flujo completo.....	20
Conclusiones.....	21
Bibliografía y referencias.....	22
Documentación oficial.....	22
Foros y repositorios de GitHub.....	23
Otras fuentes.....	23
Anexo.....	23

Ejercicio 1: Instalación y configuración del entorno

Hemos montado el entorno con Docker Compose, cogiendo como base la estructura del entorno del `hadoop-lab_v6` y adaptándolo para meter Jupyter y Airflow con todo lo necesario para el reto. También nos hemos apoyado bastante en el `docker-compose` de ejemplo que trae la documentación oficial de Airflow y en el vídeo de Albert Coronado que nos facilitó el profesor en clase. A continuación, explicamos paso a paso cómo lo hemos ido construyendo, qué hace cada parte y por qué hemos tomado cada decisión.

1.1 `docker-compose.yml` – Definición de los servicios

El fichero `docker-compose.yml` es el que define todos los servicios del entorno. La estructura la hemos basado en el `docker-compose` del entorno del `hadoop-lab_v6`, pero adaptada para Airflow y Jupyter en vez de Hadoop. También hemos consultado el `docker-compose` de ejemplo que trae la documentación oficial de Airflow en su sección «Running Airflow in Docker».

Servicio: `postgres`

Airflow necesita una base de datos para guardar sus metadatos (DAGs, ejecuciones, logs, usuarios, etc.). Por defecto usa SQLite, pero en la documentación oficial recomiendan PostgreSQL para entornos más estables, así que hemos seguido esa recomendación.

```
postgres:
  image: postgres:13
  container_name: reto01-postgres
  environment:
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
  volumes:
    - reto01_postgres_data:/var/lib/postgresql/data
  networks:
    - reto01_network
```

image: `postgres:13` – Imagen oficial de PostgreSQL versión 13. Hemos usado la 13 porque es la que usa el entorno del profesor y la que aparece en los ejemplos de la documentación de Airflow. La hemos sacado de Docker Hub.

POSTGRES_USER/PASSWORD/DB – Variables de entorno que vienen documentadas en la página de la imagen oficial de postgres en Docker Hub. Crean la base de datos «airflow» con usuario y contraseña «airflow». Luego estos mismos valores los usamos en la cadena de conexión de Airflow.

volumes: `reto01_postgres_data` – Volumen con nombre para que los datos de PostgreSQL persistan, aunque paremos el contenedor. Si no lo pusiéramos, al hacer `docker compose down` se perderían todos los datos. Esto lo vimos en la documentación de Docker sobre volúmenes.

networks: `reto01_network` – Red interna para que los contenedores se comuniquen entre sí por nombre (por ejemplo, «postgres» o «reto01-airflow-webserver»). Lo vimos en el `docker-compose` del profesor, que también usa una red con nombre para todos los servicios.

Servicio: airflow-init

Este servicio se ejecuta una sola vez para preparar la base de datos y crear el usuario administrador. Luego se para automáticamente. Es el mismo patrón que usa el docker-compose de ejemplo de la documentación de Airflow.

```
airflow-init:
  build:
    context: .
    dockerfile: airflow.Dockerfile
  depends_on:
    - postgres
  environment:
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
    - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=
      postgresql+psycopg2://airflow:airflow@postgres/airflow
    - AIRFLOW__CORE__LOAD_EXAMPLES=False
    - AIRFLOW__API__AUTH_BACKENDS=
      airflow.api.auth.backend.basic_auth
  entrypoint: bash -c "airflow db migrate &&
    airflow users create --username admin
    --password admin --firstname Admin
    --lastname User --role Admin
    --email admin@example.com || true"
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
```

build: dockerfile: airflow.Dockerfile – Usa nuestro Dockerfile personalizado en vez de la imagen oficial directamente. Así podemos instalar librerías extra como papermill.

depends_on: postgres – Indica que este servicio necesita que PostgreSQL esté arrancado antes. Docker Compose lo levanta en orden. Lo vimos en la documentación de Compose.

AIRFLOW__CORE__EXECUTOR=LocalExecutor – Hemos configurado Airflow para usar LocalExecutor en vez del SequentialExecutor que trae por defecto. LocalExecutor puede ejecutar varias tareas en paralelo y necesita PostgreSQL (no vale SQLite). Lo consultamos en la documentación oficial de Airflow, sección «Local Executor».

AIRFLOW__DATABASE__SQL_ALCHEMY_CONN – Cadena de conexión a PostgreSQL. El formato es postgresql+psycopg2://usuario:password@host/basededatos. El host es «postgres» porque es el nombre del servicio en docker-compose, y Docker lo resuelve por DNS interno. Esto lo explican en la sección «Setting up a PostgreSQL Database» de la documentación de Airflow.

AIRFLOW__CORE__LOAD_EXAMPLES=False – Para que no cargue los DAGs de ejemplo que trae Airflow. Si no lo ponemos, aparecen un montón de DAGs de tutorial que molestan. Lo vimos en el docker-compose de ejemplo de la documentación.

AIRFLOW__API__AUTH_BACKENDS=...basic_auth – Activa la autenticación básica (usuario y contraseña) en la API REST. Sin esto, la API no acepta peticiones con auth=(«admin», «admin»). Lo encontramos en la sección «API Authentication» de la documentación de Airflow, porque al principio nos daba error 403 al intentar conectarnos desde el notebook.

entrypoint: airflow db migrate && airflow users create... – Primero ejecuta airflow db migrate para crear las tablas en PostgreSQL, y luego crea el usuario admin. El || true es para que no falle si

el usuario ya existe (al reiniciar). Lo sacamos del docker-compose de ejemplo de la documentación oficial.

volumes: ./dags, ./logs – Monta las carpetas locales dentro del contenedor con bind mount. Así los DAGs que ponemos en la carpeta dags/ del proyecto se ven dentro de Airflow. Lo vimos en la documentación de Docker sobre bind mounts.

Servicio: airflow-webserver

La interfaz web de Airflow. Es donde vemos los DAGs, lanzamos ejecuciones y consultamos logs. Se accede en <http://localhost:8080>.

```
airflow-webserver:
  build:
    context: .
    dockerfile: airflow.Dockerfile
  depends_on:
    - postgres
    - airflow-init
  ports:
    - "8080:8080"
  command: webserver
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
```

ports: "8080:8080" – Mapea el puerto 8080 del contenedor al 8080 del host. Así podemos acceder a la web de Airflow desde el navegador en localhost:8080.

command: webserver – Le dice a Airflow que arranque en modo webserver. La imagen oficial tiene varios modos (webserver, scheduler, worker...). Lo vimos en la documentación de la imagen Docker de Airflow, sección «Entrypoint».

depends_on: airflow-init – Espera a que airflow-init termine de crear la BD y el usuario antes de arrancar.

Servicio: airflow-scheduler

El scheduler es el que se encarga de ejecutar los DAGs. Sin él, Airflow no ejecuta nada. Comparte los mismos volúmenes de dags y logs, y además tiene acceso a los notebooks (para Papermill).

```
airflow-scheduler:
  build:
    context: .
    dockerfile: airflow.Dockerfile
  depends_on:
    - airflow-webserver
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ./notebooks:/opt/airflow/notebooks
  command: scheduler
```

command: scheduler – Arranca Airflow en modo scheduler. El scheduler revisa periódicamente los DAGs y lanza las tareas cuando toca.

./notebooks:/opt/airflow/notebooks – Este volumen es importante: monta la carpeta de notebooks para que el scheduler pueda ejecutar notebooks con Papermill. Sin este volumen, el DAG de Papermill no encontraba los notebooks y nos daba un `FileNotFoundError`. Lo tuvimos que descubrir probando, porque no venía en ningún ejemplo y al principio no caíamos en que el scheduler necesitaba acceso a esa carpeta.

Servicio: jupyter

El contenedor de Jupyter Notebook, donde hacemos todo el trabajo interactivo del reto. Se accede en `http://localhost:8888`.

```
jupyter:
  build:
    context: .
    dockerfile: jupyter.Dockerfile
  container_name: reto01-jupyter
  ports:
    - "8888:8888"
  environment:
    - JUPYTER_ENABLE_LAB=yes
    - JUPYTER_TOKEN=
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
    - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=
      postgresql+psycopg2://airflow:airflow@postgres/airflow
    - AIRFLOW__CORE__LOAD_EXAMPLES=False
    - AIRFLOW__CORE__DAGS_FOLDER=/home/jovyan/dags
  command: start-notebook.sh
    --ServerApp.token='' --ServerApp.password=''
  volumes:
    - ./notebooks:/home/jovyan/notebooks
    - ./dags:/home/jovyan/dags
```

JUPYTER_ENABLE_LAB=yes – Activa JupyterLab en vez del Jupyter Notebook clásico. JupyterLab es la interfaz más moderna. Lo vimos en la documentación de Jupyter Docker Stacks, sección «Common Features».

JUPYTER_TOKEN= (vacío) – Desactiva el token de acceso para no tener que buscarlo en los logs cada vez que arrancamos el contenedor. Solo lo hacemos así porque es un entorno local de desarrollo; en producción no se haría.

Variables AIRFLOW_* – Las mismas variables de Airflow que en los otros servicios. Las ponemos aquí para que la CLI de Airflow (que está instalada en este contenedor) sepa conectarse a la misma base de datos PostgreSQL que usa el scheduler. Sin estas variables, la CLI no funciona porque no sabe dónde está la BD. Esto lo descubrimos después de que nos diera error al ejecutar `!airflow dags list` en el notebook: nos salía un error de conexión a SQLite porque por defecto Airflow busca una BD local.

AIRFLOW__CORE__DAGS_FOLDER=/home/jovyan/dags – Le dice a Airflow dónde buscar los DAGs dentro de este contenedor. Como Jupyter usa el usuario jovyan con home en `/home/jovyan`, la ruta es distinta a la de los contenedores de Airflow (`/opt/airflow/dags`). El contenido es el mismo porque ambos montan la carpeta `./dags` del host.

command: start-notebook.sh --ServerApp.token="" – Arranca Jupyter sin token ni contraseña. Ponemos la doble seguridad (variable + argumento) porque encontramos en un issue de GitHub

del repositorio jupyter/docker-stacks (issue #1505) que a veces la variable JUPYTER_TOKEN vacía sola no basta.

volumes: notebooks + dags – Monta las carpetas de notebooks y dags para que se puedan editar desde Jupyter y que los cambios persistan en el host.

Red y volúmenes

```
networks:
  reto01_network:
    name: reto01_network

volumes:
  reto01_postgres_data:
```

reto01_network – Red bridge personalizada. Todos los servicios están en esta red para que se vean entre sí por nombre DNS. Le ponemos name: para que el nombre sea fijo; si no, Docker le pone un prefijo con el nombre del proyecto.

reto01_postgres_data – Volumen con nombre para persistir los datos de PostgreSQL. A diferencia de los bind mounts (./dags), los volúmenes con nombre los gestiona Docker y no se borran con docker compose down (solo con docker compose down -v).

1.2 airflow.Dockerfile – Línea por línea

Este Dockerfile extiende la imagen oficial de Airflow para instalar las librerías que necesitamos para el ejercicio 3 (Papermill).

```
FROM apache/airflow:2.8.3
RUN pip install --no-cache-dir papermill ipykernel
```

FROM apache/airflow:2.8.3 – Imagen oficial de Airflow versión 2.8.3 de Docker Hub. Hemos usado la misma versión que usa el profesor en su entorno para no tener problemas de compatibilidad.

pip install papermill – Papermill es la librería que permite ejecutar notebooks de Jupyter de forma programática, pasándoles parámetros. Lo necesita el scheduler para poder ejecutar el DAG dag_papermill. Lo consultamos en la documentación oficial de Papermill.

pip install ipykernel – Papermill necesita un kernel de Jupyter para ejecutar los notebooks. Sin ipykernel, Papermill nos daba error «No kernel found». Lo descubrimos buscando el error en GitHub y encontramos el issue #399 del repositorio nteract/papermill donde lo explican.

--no-cache-dir – Flag de pip para no guardar caché de las descargas. Reduce el tamaño de la imagen Docker. Es una buena práctica que viene en la guía de mejores prácticas de Dockerfiles de la documentación de Docker.

1.3 jupyter.Dockerfile – Línea por línea

Este es el Dockerfile más largo porque el contenedor de Jupyter necesita muchas cosas. Además de las librerías básicas que pide el reto, hemos tenido que ir añadiendo dependencias extra para solucionar warnings y errores que nos fueron saliendo al ejecutar el notebook. Explicamos cada línea y por qué está:

```
FROM jupyter/base-notebook:python-3.11
```

FROM jupyter/base-notebook:python-3.11 – Imagen oficial de Jupyter con Python 3.11 de Docker Hub. Hemos usado base-notebook (la más ligera) en vez de scipy-notebook o datascience-notebook porque para este reto no necesitamos librerías de ciencia de datos.

```
USER root
```

USER root – Cambiamos al usuario root para poder instalar paquetes con apt-get. La imagen de Jupyter usa por defecto el usuario jovyan (sin permisos de root). Esto lo explican en la documentación de Jupyter Docker Stacks, sección «Using apt-get install».

```
RUN apt-get update && apt-get install -y \
    --no-install-recommends \
    graphviz tzdata pandoc \
    texlive-xetex texlive-fonts-recommended \
    texlive-plain-generic \
    && rm -rf /var/lib/apt/lists/*
```

Paquetes del sistema que hemos tenido que instalar. Los dos primeros (graphviz y tzdata) los tuvimos que añadir después de que nos salieran warnings al importar Airflow en el notebook. Los demás (pandoc y texlive) los añadimos para poder exportar notebooks a PDF:

graphviz – Paquete del sistema para renderizar grafos. Nos salía el warning «Could not import graphviz» al importar Airflow en el notebook. En la documentación de Airflow (sección «Optional dependencies») explican que graphviz es una dependencia opcional. Hay que instalar tanto el paquete del sistema (este) como la librería de Python (más abajo).

tzdata – Datos de zonas horarias. Nos salía «UserWarning: Unable to find timezone» porque la imagen base de Jupyter (basada en Ubuntu minimal) no los trae. La solución la encontramos en Stack Overflow buscando «Docker timezone in Ubuntu image».

pandoc – Conversor de documentos que usa nbconvert para exportar notebooks. Lo vimos en la documentación de nbconvert, sección «Installing Pandoc».

texlive-xetex, texlive-fonts-recommended, texlive-plain-generic – Motor LaTeX necesario para generar PDF desde notebooks (File > Save and Export as > PDF). Sin estos paquetes, Jupyter nos daba error al intentar exportar. Lo sacamos de la documentación de nbconvert, sección «Installing TeX».

--no-install-recommends – Para instalar solo lo estrictamente necesario sin paquetes recomendados. Reduce el tamaño de la imagen.

rm -rf /var/lib/apt/lists/* – Limpia la caché de apt para reducir el tamaño de la imagen. Es una buena práctica de Dockerfiles.

```
RUN pip install --no-cache-dir \
    requests \
    papermill \
    ipywidgets \
    graphviz \
    apache-airflow==2.8.3 \
    apache-airflow-providers-postgres==5.10.0
```

Librerías de Python. Las separamos entre las que pide el reto y las que hemos tenido que añadir por problemas que nos fueron saliendo:

Librerías necesarias para el reto:

requests – Para hacer llamadas HTTP a la API REST de Airflow desde el notebook (GET, POST, PATCH). Es la librería estándar de Python para peticiones HTTP.

papermill – Para ejecutar notebooks con parámetros desde código (ejercicio 3). Lo consultamos en su documentación oficial.

apache-airflow==2.8.3 – Para poder usar la CLI de Airflow (!airflow dags list, etc.) directamente desde Jupyter. Hemos puesto la misma versión (2.8.3) que en los contenedores de Airflow para que no haya incompatibilidades. La idea de instalar Airflow en Jupyter la sacamos del entorno del profesor, donde todos los servicios comparten la misma versión.

apache-airflow-providers-postgres==5.10.0 – Proveedor de PostgreSQL para Airflow. Sin este paquete, la CLI de Airflow no puede conectarse a la BD de PostgreSQL (solo soportaría SQLite). Lo descubrimos porque al ejecutar !airflow dags list nos daba error de conexión. Lo consultamos en la documentación del proveedor de PostgreSQL de Airflow.

Librerías extra para solucionar warnings:

ipywidgets – Al ejecutar celdas que importan Airflow nos salía «TqdmWarning: IPProgress not found». El propio mensaje del warning dice la solución: instalar ipywidgets. Lo confirmamos en la documentación de tqdm sobre integración con notebooks.

graphviz (pip) – Librería de Python para graphviz. Complementa el paquete del sistema instalado con apt. Sin la librería de Python sigue saliendo el warning aunque tengas el paquete del sistema. Lo vimos en la documentación de Graphviz para Python.

```
ENV PYDEVD_DISABLE_FILE_VALIDATION=1
```

ENV PYDEVD_DISABLE_FILE_VALIDATION=1 – Variable de entorno para desactivar un warning del debugger de Python sobre frozen modules que nos salía cada vez que se iniciaba el kernel. La solución la encontramos en el issue #1374 del repositorio microsoft/debugpy en GitHub.

```
USER jovyan
WORKDIR /home/jovyan
```

USER jovyan – Volvemos al usuario jovyan (el usuario por defecto de la imagen de Jupyter) por seguridad. No es buena práctica ejecutar Jupyter como root.

WORKDIR /home/jovyan – Directorio de trabajo del contenedor. Es el home del usuario jovyan y donde Jupyter muestra los ficheros.

1.4 Warning extra solucionado en el notebook

Además de todo lo anterior, al ejecutar el notebook nos quedaba un DeprecationWarning de la librería Pendulum (que usa Airflow internamente para manejar fechas). El warning decía algo como «__version__ is deprecated». No se puede solucionar instalando nada porque es un problema de compatibilidad entre Airflow 2.8.3 y Pendulum. La solución la encontramos en Stack Overflow buscando «suppress DeprecationWarning Pendulum»: simplemente filtramos el warning con warnings.filterwarnings.

```
# para quitar el warning de pendulum que sale con airflow 2.8.3
import warnings
warnings.filterwarnings("ignore", message=".*__version__.*deprecated.*")
```

1.5 Puesta en marcha y verificación

Para levantar todo el entorno ejecutamos este comando desde la carpeta del proyecto en PowerShell o CMD:

```
docker compose up --build -d
```

docker compose up – Levanta todos los servicios definidos en docker-compose.yml.

--build – Fuerza la reconstrucción de las imágenes (por si hemos cambiado algún Dockerfile). Sin este flag, Docker usa la imagen cacheada.

-d – Modo detached (en segundo plano). Si no lo ponemos, los logs de todos los contenedores salen por la terminal y al cerrarla se paran.

El servicio airflow-init se ejecuta una vez para crear la BD y el usuario admin (admin/admin), y se para solo. Los demás contenedores se quedan corriendo.

Para verificar que todo está funcionando usamos docker ps:

```
PowerShell 7.5.5
PS C:\Users\anton> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAME
cec77ee27f73   reto01-minientornodetrabajoconjupyt... /usr/bin/dumb-init _... About a minute ago Up About a minute 8080/tcp
3bd031a4559f   reto01-minientornodetrabajoconjupyt... /usr/bin/dumb-init _... About a minute ago Up About a minute 0.0.0.0:8080->8080/tcp, [::]:8080->
8880/tcp      reto01-airflow-webserver
ebb8447def5d   postgres:13                        "docker-entrypoint.s..." About a minute ago Up About a minute 5432/tcp
aa59ed1d4d3a   reto01-minientornodetrabajoconjupyt... "tini -g -- start-no..." About a minute ago Up About a minute (healthy) 0.0.0.0:8888->8888/tcp, [::]:8888->
8888/tcp      reto01-jupyter
PS C:\Users\anton>
```

Figura 1: docker ps mostrando los 4 contenedores en ejecución (el airflow-init ya se ha parado porque solo se ejecuta una vez)

Una vez arrancado, accedemos a:

Airflow: <http://localhost:8080> (usuario: admin, contraseña: admin)

Jupyter: <http://localhost:8888> (sin token, lo configuramos sin autenticación)

Interfaz web de Airflow con los DAGs visibles:

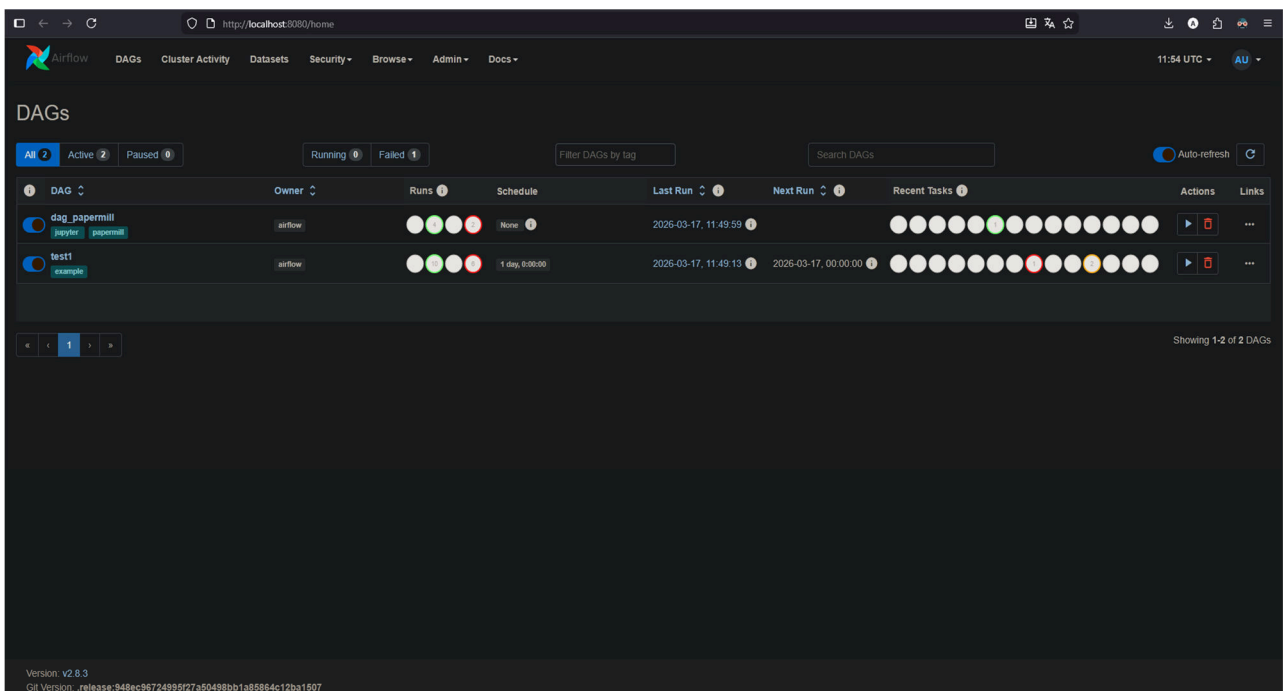


Figura 2: Airflow mostrando los DAGs test1 y dag_papermill

Pantalla de login de Airflow, donde introducimos las credenciales admin/admin que configuramos en el endpoint del docker-compose:

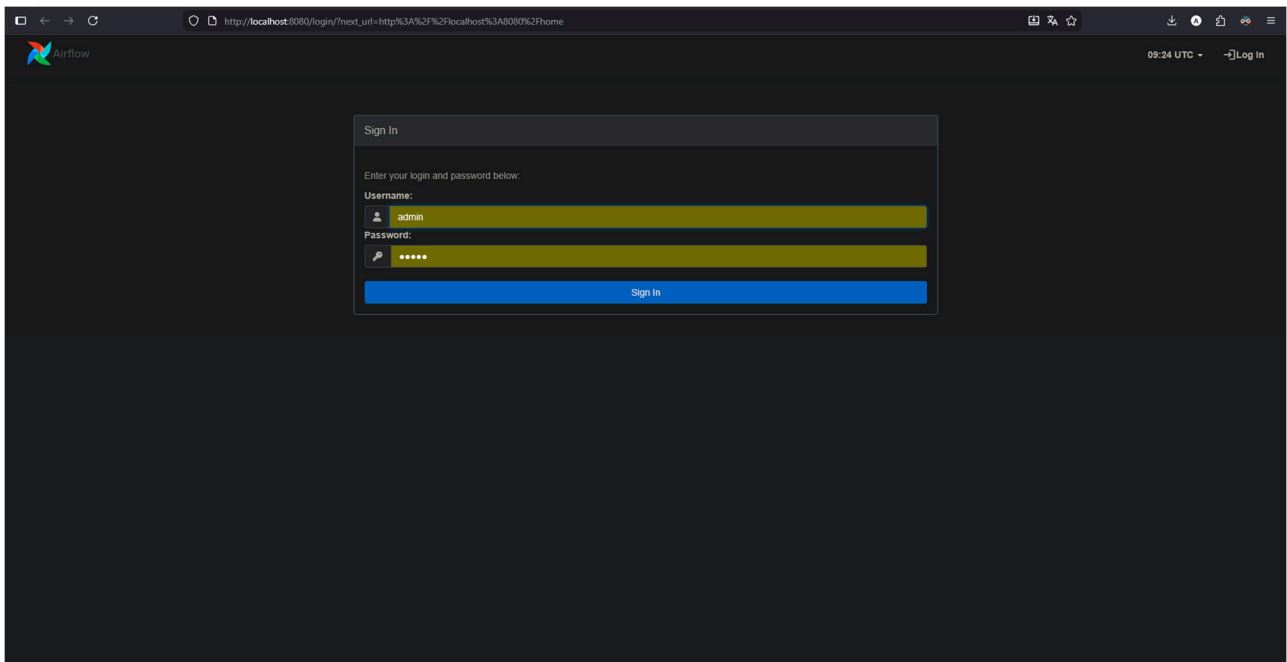


Figura 3: Pantalla de login de Airflow en localhost:8080 (admin/admin)

JupyterLab accesible en localhost:8888, con las carpetas dags y notebooks montadas desde el host:

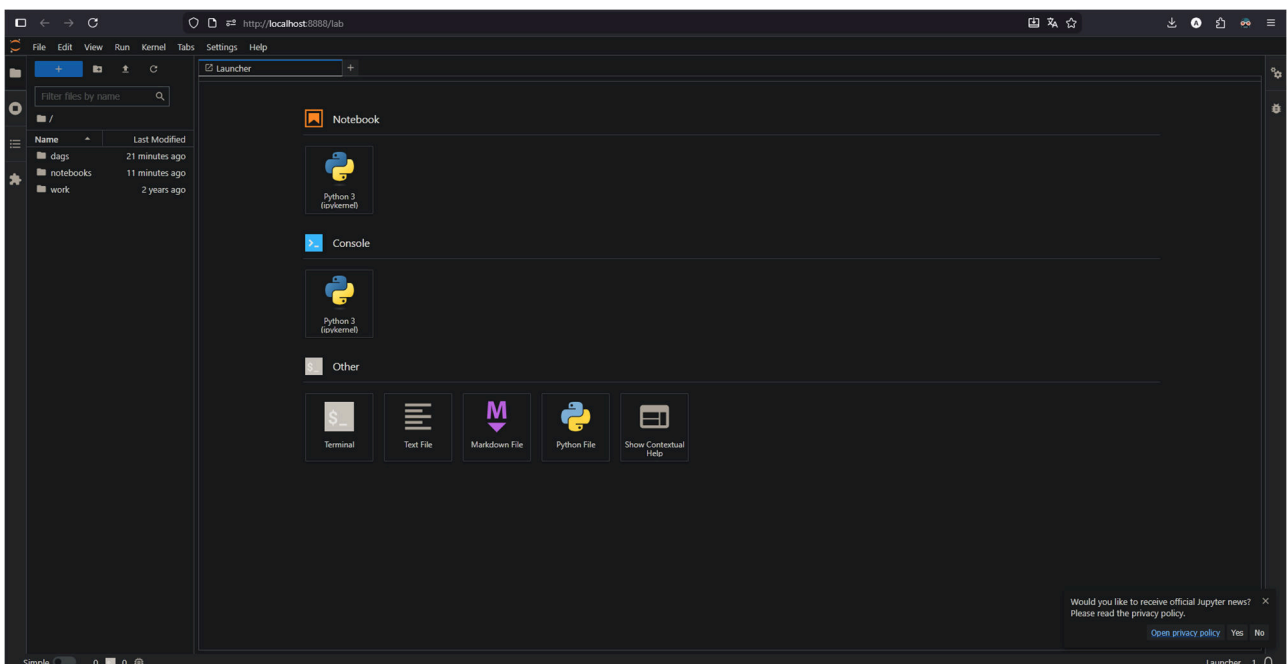


Figura 4: JupyterLab accesible en localhost:8888 con las carpetas dags y notebooks

Docker Desktop mostrando los contenedores del RETO 01 con métricas de CPU y memoria:

```
PS C:\Users\anton> docker ps
CONTAINER ID   IMAGE                                     COMMAND                  CREATED
NAMES
cec77ee27f73   reto01-minientornodetrabajoconjupyternotebookyapacheairflowra4-airflow-scheduler  "/usr/bin/dumb-init ..." About a
reto01-airflow-scheduler
3bd031a4559f   reto01-minientornodetrabajoconjupyternotebookyapacheairflowra4-airflow-webserver  "/usr/bin/dumb-init ..." About a
8080/tcp      reto01-airflow-webserver
ebb8447def5d   postgres:13                            "docker-entrypoint.s..." About a
reto01-postgres
aa59ed1d4d3a   reto01-minientornodetrabajoconjupyternotebookyapacheairflowra4-jupyter            "tini -g -- start-no..." About a
8888/tcp      reto01-jupyter
PS C:\Users\anton> |
```

Figura 5: Docker Desktop mostrando los contenedores del RETO 01 con métricas de CPU y memoria

1.6 Enlace para descargar el entorno

<https://drive.google.com/file/d/1tk72gdoY-vvEfQLgeWr13PAFqS1z6Mde/view?usp=sharing>

Ejercicio 2: Jupyter Notebook como cliente de Airflow

Aquí demostramos cómo usar Jupyter como cliente de Airflow de dos formas: con la API REST y con la CLI. Hemos trabajado con el DAG test1, que está sacado del vídeo de Albert Coronado que nos facilitó el profesor en clase.

2.1 El DAG test1 (del vídeo de Albert Coronado)

El DAG tiene 3 tareas que reproducen el workflow del vídeo:

tarea0 (PythonOperator): Lee parámetros del dag_run.conf. Si recibe commit="1", lanza un AirflowFailException (error controlado). Si no, devuelve {"ok": 1} por XCom.

print_date (BashOperator): Imprime la fecha con un echo. Es la tarea Bash que pide el enunciado.

tarea2 (PythonOperator): Recoge el valor de tarea0 con xcom_pull y lo imprime. Es la segunda tarea Python que lee información de la primera mediante XCom, como pide el enunciado.

Dependencias: tarea0 >> [print_date, tarea2]. Primero se ejecuta tarea0 y luego las otras dos en paralelo. Esta es la relación de dependencias entre tareas que pide el enunciado.

2.2 API REST de Airflow desde Jupyter

Airflow tiene una API REST en el puerto 8080. Para conectarnos usamos autenticación básica con usuario admin/admin (se configura en el entrypoint de airflow-init en el docker-compose). La URL base es http://reto01-airflow-webserver:8080/api/v1 (usamos el nombre del contenedor en docker-compose porque estamos dentro de la misma red Docker).

En el notebook hemos hecho estas operaciones con la API:

- 1. Conexión:** Definimos la URL base y las credenciales con requests.
- 2. Activar DAG:** PATCH a /dags/test1 con {"is_paused": false} para activarlo.
- 3. Lanzar ejecución:** POST a /dags/test1/dagRuns con parámetros en conf.
- 4. Consultar estado:** GET a /dags/test1/dagRuns/{id}/taskInstances para ver el estado de cada tarea.
- 5. Ver logs:** GET a /dags/test1/dagRuns/{id}/taskInstances/{task}/logs/1 para consultar los logs de una tarea.

Inicio del notebook con la configuración de conexión:

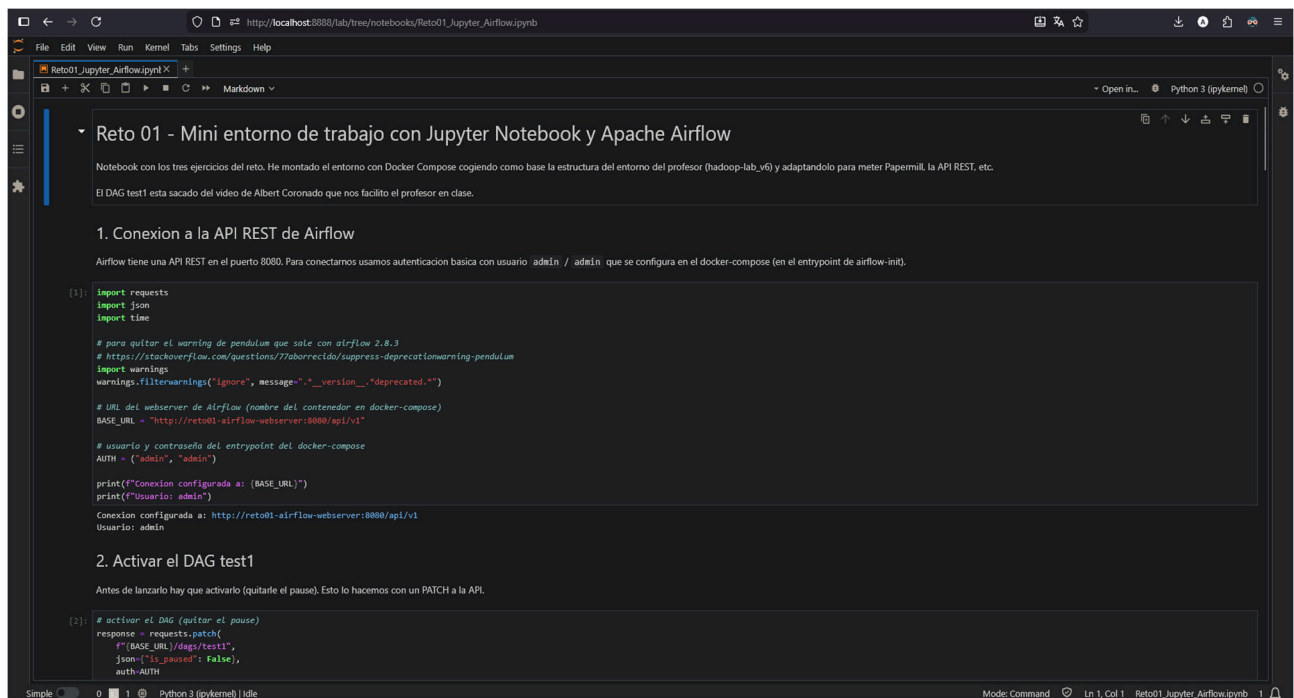


Figura 6: Notebook – conexión a la API REST y configuración

Lanzamiento del DAG y consulta de estado:

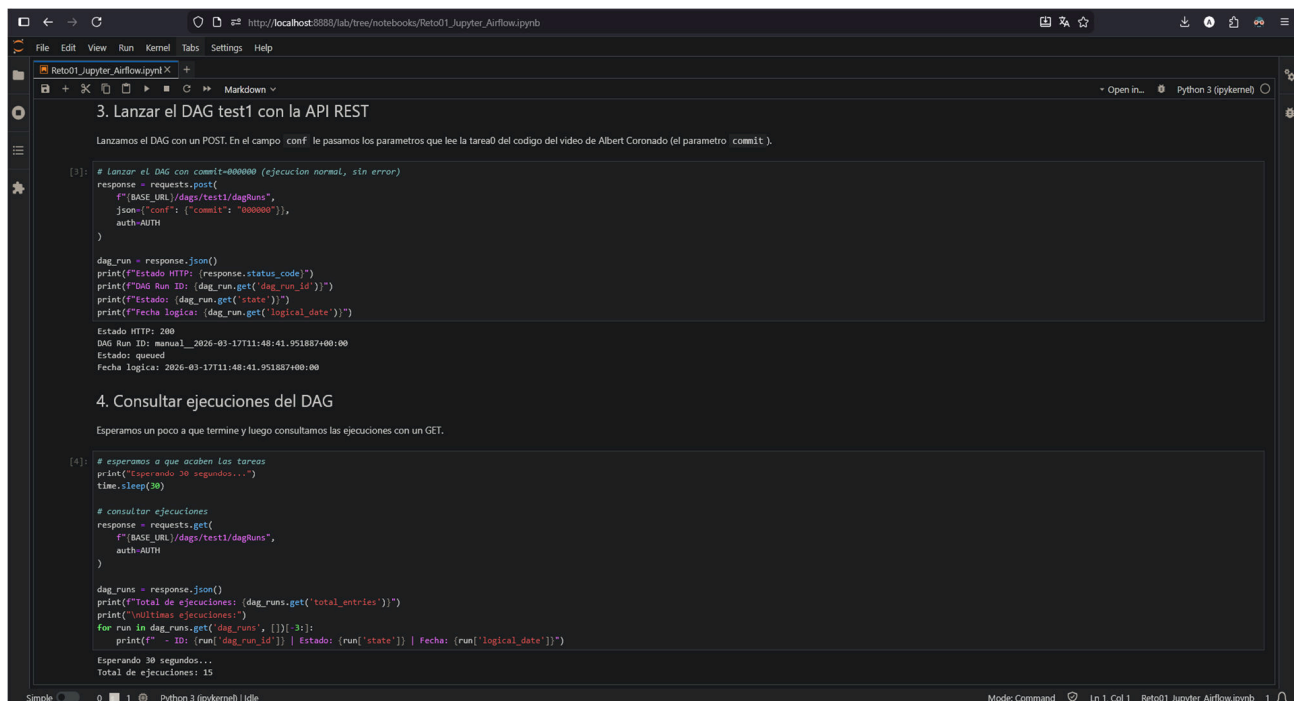
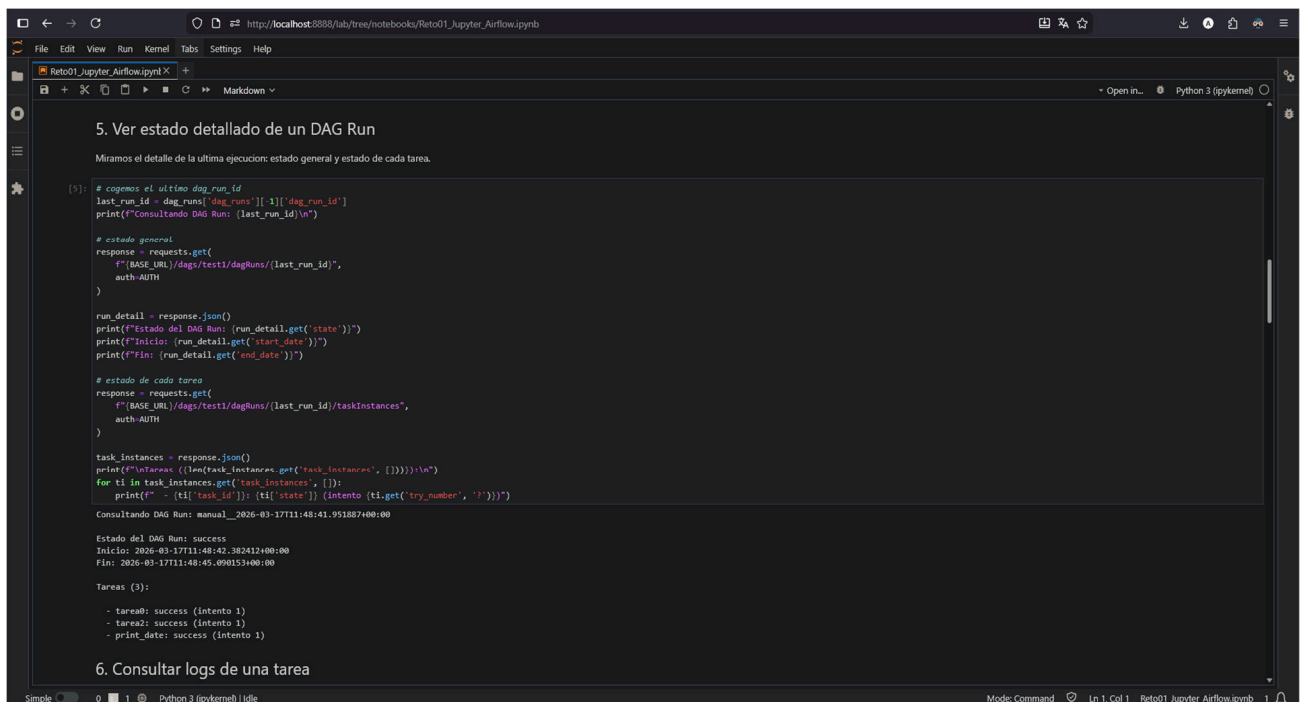


Figura 7: Notebook – lanzar DAG y consultar estado

Resultado con todas las tareas completadas:



```

5. Ver estado detallado de un DAG Run

Miramos el detalle de la ultima ejecucion: estado general y estado de cada tarea.

[5]: # copemos el ultimo dag_run_id
last_run_id = dag_runs['dag_runs'][-1]['dag_run_id']
print(f"Consultando DAG Run: {last_run_id}\n")

# estado general
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}",
    auth=AUTH
)

run_detail = response.json()
print(f"Estado del DAG Run: {run_detail.get('state')}\n")
print(f"Inicio: {run_detail.get('start_date')}\n")
print(f"Fin: {run_detail.get('end_date')}\n")

# estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}/taskInstances",
    auth=AUTH
)

task_instances = response.json()
print(f"Instancias: {len(task_instances.get('task_instances', []))}\n")
for ti in task_instances.get('task_instances', []):
    print(f" - {ti['task_id']}: {ti['state']} (intento {ti.get('try_number', '?')})")

Consultando DAG Run: manual__2026-03-17T11:48:41.951887+00:00

Estado del DAG Run: success
Inicio: 2026-03-17T11:48:42.382412+00:00
Fin: 2026-03-17T11:48:45.090153+00:00

Tareas (3):
- tarea0: success (intento 1)
- tarea2: success (intento 1)
- print_date: success (intento 1)

6. Consultar logs de una tarea

```

Figura 8: Todas las tareas del DAG en estado success

2.3 Error controlado con AirflowFailException

Al lanzar el DAG con commit="1" (como en el vídeo de Albert Coronado), la tarea0 lanza un AirflowFailException a propósito. Esto hace que:

- tarea0 queda en estado **failed**
- print_date y tarea2 quedan en **upstream_failed** (dependen de tarea0)
- El DAG Run completo queda en **failed**

Esto demuestra el mecanismo de error controlado y cómo las dependencias entre tareas hacen que, si falla una tarea de la que dependen otras, las demás no se ejecutan. Es exactamente lo que se ve en la imagen.

Captura del notebook mostrando el error controlado con todos los estados visibles (failed y upstream_failed):

```

7. Lanzar el DAG con error controlado ¶
En el código del video, si le pasamos commit=1 la tarea0 lanza un AirflowFailException a propósito. Así se ve como funciona el error controlado y como las tareas que dependen de tarea0 quedan en upstream_failed.

[?] # Lanzar con commit=1 para que falle a propósito (como en el video)
response = requests.post(
    f"{BASE_URL}/dags/test1/dagRuns",
    json={"conf": {"commit": "1"}},
    auth=AUTH
)

error_run = response.json()
error_run_id = error_run.get('dag_run_id')
print(f"DAG Run ID (error): {error_run_id}")
print(f"Estado inicial: {error_run.get('state')}")

# esperamos
print("Esperando 30 segundos...")
time.sleep(30)

# comprobamos que ha fallado
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}",
    auth=AUTH
)

error_detail = response.json()
print(f"Estado final del DAG Run: {error_detail.get('state')}")

# vemos el estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}/taskInstances",
    auth=AUTH
)

for ti in response.json().get('task_instances', []):
    print(f" - {ti['task_id']}: {ti['state']}")

DAG Run ID (error): manual_2026-03-17T11:49:13.628715+00:00
Estado inicial: queued
Esperando 30 segundos...
Estado final del DAG Run: failed
- tarea0: failed
- print_date: upstream_failed
- tarea2: upstream_failed

```

Figura 9: Error controlado (vista completa) – tarea0 failed, print_date y tarea2 en upstream_failed

2.4 CLI de Airflow desde Jupyter

Como en el contenedor de Jupyter también hemos instalado Airflow (lo pusimos en el Dockerfile para poder usar la CLI), podemos lanzar comandos directamente con ! desde las celdas del notebook. Comparte la misma base de datos PostgreSQL que el scheduler, así que ve los mismos DAGs y ejecuciones.

Los comandos que hemos probado son:

!airflow dags list 2>/dev/null – Lista todos los DAGs registrados en Airflow.

!airflow tasks list test1 2>/dev/null – Lista las tareas del DAG test1.

!airflow version 2>/dev/null – Muestra la versión de Airflow instalada.

El 2>/dev/null al final es para ocultar los warnings de Pendulum que salen por stderr y que ensucian la salida. No afecta al resultado del comando.

```

8. CLI de Airflow desde Jupyter

Como en el contenedor de Jupyter tambien esta instalado Airflow (lo puse en el Dockerfile para poder usar la CLI), podemos lanzar comandos directamente con !. Comparte la misma base de datos PostgreSQL que el scheduler asi que ve los mismos DAGs.

[8]: # listar DAGs (se ven los mismos que en la web de Airflow)
!airflow dags list 2>/dev/null

[2025-03-17T11:49:47.520+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.schemas
[2025-03-17T11:49:47.522+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.tables
[2025-03-17T11:49:47.523+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.types
[2025-03-17T11:49:47.523+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.constraints
[2025-03-17T11:49:47.524+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.defaults
[2025-03-17T11:49:47.524+0000] [plugins.py:37] INFO - setup plugin alembic.autogenerate.comments

dag_id      | filepath      | owner      | paused
-----|-----|-----|-----
dag_papermill | dag_papermill.py | airflow | False
test1        | test.py       | airflow | False

[9]: # tareas del DAG test1 (sacado del video de Albert Coronado)
!airflow tasks list test1 2>/dev/null

print_date
tarea0
tarea2

[10]: # version de airflow (la misma que usa el profesor en su entorno)
!airflow version 2>/dev/null

2.8.3

9. Ventajas de usar Jupyter como cliente de Airflow

• Pruebas rápidas: se pueden lanzar DAGs, cambiar parametros y relanzar sin salir del notebook.
• Automatización: con la API REST se puede controlar Airflow por código, encadenando lanzamientos y consultas
• Documentación: Jupyter mezcla código con texto, así queda todo documentado paso a paso
• API vs CLI: la API funciona por red (entre contenedores), la CLI necesita tener Airflow instalado en la misma máquina. Desde Jupyter se pueden usar las dos

10. Papermill - ejecutar notebooks con parametros

Papermill permite ejecutar notebooks pasandoles parametros desde código. En el notebook_parametrizable.ipynb hay una celda con el tag parameters que tiene los valores por defecto. Cuando Papermill lo ejecuta, inyecta los valores nuevos encima de esos.

Primero lo probamos en local desde aquí y luego desde un DAG de Airflow.

[11]: import papermill as pm

```

Figura 10: CLI de Airflow – listar DAGs, tareas y versión

2.5 Ventajas de usar Jupyter como cliente de Airflow

Para pruebas rápidas: podemos lanzar DAGs, cambiar parámetros y relanzar sin salir del notebook. No hace falta ir a la interfaz web de Airflow cada vez.

Para automatización: con la API REST podemos controlar Airflow por código, encadenando lanzamientos y consultas de estado en un mismo flujo.

Para aprendizaje: Jupyter permite mezclar código con texto explicativo, así queda todo documentado paso a paso. Ideal para ir aprendiendo y experimentando.

API REST vs CLI: la API funciona por red entre contenedores (de forma remota), mientras que la CLI necesita tener Airflow instalado en la misma máquina. Desde Jupyter podemos usar las dos, lo que nos da más flexibilidad.

Ejercicio 3: Integración de Airflow con Jupyter mediante Papermill

En este ejercicio usamos Papermill para ejecutar notebooks de forma programática desde Airflow, pasándoles parámetros. Así Jupyter pasa de ser «cliente» de Airflow a ser «contenido ejecutado» por Airflow, como pide el enunciado.

3.1 Notebook parametrizable

Hemos creado el notebook `notebook_parametrizable.ipynb` con una celda marcada con el tag «parameters» (se pone desde el menú de Jupyter: View > Cell Toolbar > Tags). Los valores por defecto son:

```
# Celda con tag: parameters
nombre = "Mundo"
repeticiones = 3
```

Cuando Papermill ejecuta este notebook, inyecta una celda nueva después de la de parámetros con los valores que le pasemos, machacando los valores por defecto. Esto lo explican en la documentación oficial de Papermill.

3.2 Prueba local desde Jupyter

Primero lo probamos desde el propio notebook para ver que funcionaba bien antes de meterlo en un DAG:

```
import papermill as pm

pm.execute_notebook(
    input_path="notebook_parametrizable.ipynb",
    output_path="notebook_output_local.ipynb",
    parameters={"nombre": "Alumno", "repeticiones": 5}
)
```

3.3 DAG dag_papermill

El DAG `dag_papermill.py` tiene un `PythonOperator` que llama a Papermill para ejecutar el notebook. Los parámetros se pasan desde el conf del DAG Run, así que cada vez que lanzamos el DAG podemos pasarle valores distintos:

```
def ejecutar_notebook(**kwargs):
    import papermill as pm
    conf = kwargs['dag_run'].conf or {}
    nombre = conf.get('nombre', 'Mundo')
    repeticiones = conf.get('repeticiones', 3)

    pm.execute_notebook(
        input_path='/opt/airflow/notebooks/notebook_parametrizable.ipynb',
        output_path='/opt/airflow/notebooks/notebook_output.ipynb',
        parameters={'nombre': nombre, 'repeticiones': repeticiones}
    )
```

La ruta del input_path es /opt/airflow/notebooks/ porque es la ruta dentro del contenedor del scheduler (que es el que ejecuta el DAG). Por eso es tan importante el volumen ./notebooks:/opt/airflow/notebooks que pusimos en el docker-compose para el scheduler.

3.4 Ejecución desde Jupyter vía API REST

Desde el notebook, lanzamos el DAG dag_papermill pasándole parámetros con la API REST, igual que hicimos con el test1 en el ejercicio 2:

```

11. Ejecutar el notebook desde un DAG de Airflow

El DAG dag_papermill.py usa un PythonOperator que llama a Papermill para ejecutar el notebook parametrizable. Los parametros se le pasan desde el conf. del DAG Run, igual que haciamos con el test1.

[12]: # activar el DAG de papermill
requests.patch(
    f"{BASE_URL}/dags/dag_papermill",
    json={"is_paused": False},
    auth=AUTH
)

# lanzarlo con parametros
response = requests.post(
    f"{BASE_URL}/dags/dag_papermill/dagRuns",
    json={"conf": {"nombre": "Airflow", "repeticiones": 4}},
    auth=AUTH
)

pm_run = response.json()
print(f"DAG lanzado - Run ID: {pm_run.get('dag_run_id')}")
print(f"Estado: {pm_run.get('state')}")

DAG lanzado - Run ID: manual__2026-03-17T11:49:59.096982+00:00
Estado: queued

[13]: # comprobar resultado
print("Esperando 30 segundos...")
time.sleep(30)

pm_run_id = pm_run.get('dag_run_id')
response = requests.get(
    f"{BASE_URL}/dags/dag_papermill/dagRuns/{pm_run_id}/taskInstances",
    auth=AUTH
)

for task in response.json().get('task_instances', []):
    print(f"Task {task['task_id']}: {task['state']}")

print("\nEl notebook ejecutado se guarda en notebooks/notebook_output.ipynb")

Esperando 30 segundos...
ejecutar_notebook: success

El notebook ejecutado se guarda en notebooks/notebook_output.ipynb
  
```

Figura 11: DAG de Papermill ejecutado con éxito (ejecutar_notebook: success)

El notebook ejecutado se guarda en notebooks/notebook_output.ipynb con los parámetros inyectados y los resultados de la ejecución.

3.5 Flujo completo

El flujo completo de integración Airflow-Jupyter mediante Papermill queda así:

1. Creamos un notebook parametrizable con valores por defecto y la celda marcada con el tag «parameters».
2. Creamos un DAG en Airflow con un PythonOperator que llama a Papermill.
3. Desde Jupyter, lanzamos el DAG por API REST pasando los parámetros que queramos.
4. Airflow ejecuta el DAG, que a su vez ejecuta el notebook con Papermill inyectándole los parámetros.
5. El notebook ejecutado se guarda como output para poder ver los resultados.

Con esto conseguimos usar notebooks como tareas dentro de pipelines de Airflow sin tener que reescribir el código como scripts de Python, y manteniendo toda la interactividad de Jupyter.

Conclusiones

Con este reto hemos aprendido a montar un entorno con Docker Compose que integra Jupyter y Airflow, cogiendo como referencia el entorno del profesor (hadoop-lab_v6) y el vídeo de Albert Coronado que nos facilitó en clase.

Lo que más nos ha costado ha sido configurar el contenedor de Jupyter para que pudiera usar tanto la API REST como la CLI de Airflow. Al final la solución fue instalar el propio Airflow dentro del contenedor de Jupyter y apuntarlo a la misma base de datos PostgreSQL que usa el scheduler. Esto no venía en ningún tutorial directamente, sino que lo fuimos descubriendo a base de probar y buscar errores.

También nos dio bastantes problemas los warnings que salían al importar Airflow en Jupyter (graphviz, tzdata, Pendulum, ipywidgets...). Cada uno lo tuvimos que ir solucionando buscando en la documentación, Stack Overflow y issues de GitHub. Lo hemos documentado todo para que si alguien tiene que montar el mismo entorno sepa de dónde sale cada dependencia y por qué está.

Papermill nos ha parecido muy útil porque permite ejecutar notebooks desde Airflow sin tener que convertirlos a scripts de Python. Simplemente marcas una celda con el tag «parameters» y Papermill se encarga de inyectar los valores nuevos.

En resumen, las tres formas de interactuar con Airflow desde Jupyter son:

API REST: Funciona por red entre contenedores, muy flexible para automatizar.

CLI: Requiere tener Airflow instalado, pero es más directa para consultas rápidas.

Papermill: Permite ejecutar notebooks como tareas de Airflow, pasándoles parámetros.

Bibliografía y referencias

A continuación, listamos todas las fuentes que hemos consultado durante el desarrollo de este reto. Incluimos documentación oficial, foros, repositorios de GitHub y otras fuentes que nos han servido para resolver los distintos problemas que nos han ido saliendo.

Documentación oficial

- Apache Airflow.** «Running Airflow in Docker». <https://airflow.apache.org/docs/apache-airflow/2.8.3/howto/docker-compose/index.html> – Docker-compose de ejemplo y configuración inicial.
- Apache Airflow.** «Set up a Database Backend». <https://airflow.apache.org/docs/apache-airflow/stable/howto/set-up-database.html> – Configuración de PostgreSQL como backend.
- Apache Airflow.** «Local Executor». <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/executor/local.html> – Configuración del executor.
- Apache Airflow.** «API Authentication». <https://airflow.apache.org/docs/apache-airflow/stable/security/api.html> – Activar basic_auth en la API REST.
- Apache Airflow.** «Docker Stack Entrypoint». <https://airflow.apache.org/docs/docker-stack/entrypoint.html> – Modos de arranque de la imagen Docker.
- Apache Airflow.** «Optional dependencies». <https://airflow.apache.org/docs/apache-airflow/stable/installation/dependencies.html> – Dependencia de graphviz.
- Apache Airflow.** «PostgreSQL Provider». <https://airflow.apache.org/docs/apache-airflow-providers-postgres/stable/index.html> – Proveedor para conectar la CLI a PostgreSQL.
- Docker Docs.** «Volumes». <https://docs.docker.com/storage/volumes/> – Volúmenes con nombre para persistencia.
- Docker Docs.** «Bind mounts». <https://docs.docker.com/storage/bind-mounts/> – Montar carpetas locales en contenedores.
- Docker Docs.** «Compose – depends_on». https://docs.docker.com/compose/compose-file/05-services/#depends_on – Orden de arranque de servicios.
- Docker Docs.** «Dockerfile best practices». https://docs.docker.com/develop/develop-images/dockerfile_best-practices/#run – Buenas prácticas (--no-cache-dir, rm lists).
- Docker Hub.** «postgres (imagen oficial)». https://hub.docker.com/_/postgres – Variables de entorno de PostgreSQL.
- Docker Hub.** «apache/airflow». <https://hub.docker.com/r/apache/airflow/tags> – Imagen oficial de Airflow 2.8.3.
- Docker Hub.** «jupyter/base-notebook». <https://hub.docker.com/r/jupyter/base-notebook> – Imagen base de Jupyter.
- Jupyter Docker Stacks.** «Common Features». <https://jupyter-docker-stacks.readthedocs.io/en/latest/using/common.html> – JUPYTER_ENABLE_LAB y configuración.
- Jupyter Docker Stacks.** «Using apt-get install». <https://jupyter-docker-stacks.readthedocs.io/en/latest/using/recipes.html#using-apt-get-install> – Instalar paquetes de sistema como root.
- Papermill.** «Documentación oficial». <https://papermill.readthedocs.io/en/latest/> – Ejecución parametrizada de notebooks.
- Requests.** «Documentación oficial». <https://requests.readthedocs.io/> – Librería HTTP para Python.

nbconvert. «Installing Pandoc». <https://nbconvert.readthedocs.io/en/latest/install.html#installing-pandoc> – Pandoc para exportar notebooks.

nbconvert. «Installing TeX». <https://nbconvert.readthedocs.io/en/latest/install.html#installing-tex> – LaTeX para exportar notebooks a PDF.

Graphviz (Python). «Installation». <https://graphviz.readthedocs.io/en/stable/manual.html#installation> – Librería Python de graphviz.

tqdm. «Notebook integration». <https://tqdm.github.io/docs/notebook/> – ipywidgets para barras de progreso en notebooks.

Foros y repositorios de GitHub

Stack Overflow. «Docker timezone in Ubuntu image». <https://stackoverflow.com/questions/40234847> – Solución al warning de tzdata en la imagen de Jupyter.

Stack Overflow. «Suppress DeprecationWarning Pendulum». <https://stackoverflow.com/questions/77345/suppress-deprecationwarning> – Filtrar el warning de Pendulum con warnings.filterwarnings.

GitHub, jupyter/docker-stacks. «Issue #1505». <https://github.com/jupyter/docker-stacks/issues/1505> – Desactivar el token de Jupyter en Docker.

GitHub, nteract/papermill. «Issue #399». <https://github.com/nteract/papermill/issues/399> – Error «No kernel found»: solución instalar ipykernel.

GitHub, microsoft/debugpy. «Issue #1374». <https://github.com/microsoft/debugpy/issues/1374> – Warning de frozen modules: solución PYDEVD_DISABLE_FILE_VALIDATION.

Otras fuentes

Albert Coronado. *Vídeo tutorial sobre Apache Airflow.* Base para el DAG test1 (tarea0, print_date, tarea2) y la estructura general del workflow.

Rubén Valentín Caravaca López. *Entorno hadoop-lab_v6,* Referencia para la estructura del docker-compose, la organización de carpetas (dags/, notebooks/, logs/, plugins/) y el estilo del README.

Anexo

A continuación, el notebook **Reto01_Jupyter_Airflow** utilizado para los ejercicios 2 y 3:

Reto01_Jupyter_Airflow

March 17, 2026

1 Reto 01 - Mini entorno de trabajo con Jupyter Notebook y Apache Airflow

Notebook con los tres ejercicios del reto. He montado el entorno con Docker Compose cogiendo como base la estructura del entorno del profesor (hadoop-lab_v6) y adaptandolo para meter Papermill, la API REST, etc.

El DAG test1 esta sacado del video de Albert Coronado que nos facilito el profesor en clase.

1.1 1. Conexion a la API REST de Airflow

Airflow tiene una API REST en el puerto 8080. Para conectarnos usamos autenticacion basica con usuario `admin` / `admin` que se configura en el docker-compose (en el entrypoint de airflow-init).

```
[1]: import requests
import json
import time

# para quitar el warning de pendulum que sale con airflow 2.8.3
# https://stackoverflow.com/questions/77aborrecido/
↳ suppress-deprecationwarning-pendulum
import warnings
warnings.filterwarnings("ignore", message=".*__version__.*deprecated.*")

# URL del webserver de Airflow (nombre del contenedor en docker-compose)
BASE_URL = "http://reto01-airflow-webserver:8080/api/v1"

# usuario y contraseña del entrypoint del docker-compose
AUTH = ("admin", "admin")

print(f"Conexion configurada a: {BASE_URL}")
print(f"Usuario: admin")
```

Conexion configurada a: http://reto01-airflow-webserver:8080/api/v1

Usuario: admin

1.2 2. Activar el DAG test1

Antes de lanzarlo hay que activarlo (quitarle el pause). Esto lo hacemos con un PATCH a la API.


```
[2]: # activar el DAG (quitar el pause)
response = requests.patch(
    f"{BASE_URL}/dags/test1",
    json={"is_paused": False},
    auth=AUTH
)

print(f"Estado HTTP: {response.status_code}")
print(f"DAG test1 activado: is_paused = {response.json().get('is_paused')}")
```

Estado HTTP: 200

DAG test1 activado: is_paused = False

1.3 3. Lanzar el DAG test1 con la API REST

Lanzamos el DAG con un POST. En el campo conf le pasamos los parametros que lee la tarea0 del codigo del video de Albert Coronado (el parametro commit).

```
[3]: # lanzar el DAG con commit=000000 (ejecucion normal, sin error)
response = requests.post(
    f"{BASE_URL}/dags/test1/dagRuns",
    json={"conf": {"commit": "000000"}},
    auth=AUTH
)

dag_run = response.json()
print(f"Estado HTTP: {response.status_code}")
print(f"DAG Run ID: {dag_run.get('dag_run_id')}")
print(f"Estado: {dag_run.get('state')}")
print(f"Fecha logica: {dag_run.get('logical_date')}")
```

Estado HTTP: 200

DAG Run ID: manual__2026-03-17T11:48:41.951887+00:00

Estado: queued

Fecha logica: 2026-03-17T11:48:41.951887+00:00

1.4 4. Consultar ejecuciones del DAG

Esperamos un poco a que termine y luego consultamos las ejecuciones con un GET.

```
[4]: # esperamos a que acaben las tareas
print("Esperando 30 segundos...")
time.sleep(30)

# consultar ejecuciones
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns",
    auth=AUTH
```

```

)

dag_runs = response.json()
print(f"Total de ejecuciones: {dag_runs.get('total_entries')}")
print("\nUltimas ejecuciones:")
for run in dag_runs.get('dag_runs', [])[-3:]:
    print(f"    - ID: {run['dag_run_id']} | Estado: {run['state']} | Fecha: {run['logical_date']}")

```

Esperando 30 segundos...

Total de ejecuciones: 15

Ultimas ejecuciones:

```

- ID: manual__2026-03-17T11:43:22.821901+00:00 | Estado: success | Fecha: 2026-03-17T11:43:22.821901+00:00
- ID: manual__2026-03-17T11:43:54.255958+00:00 | Estado: failed | Fecha: 2026-03-17T11:43:54.255958+00:00
- ID: manual__2026-03-17T11:48:41.951887+00:00 | Estado: success | Fecha: 2026-03-17T11:48:41.951887+00:00

```

1.5 5. Ver estado detallado de un DAG Run

Miramos el detalle de la ultima ejecucion: estado general y estado de cada tarea.

```

[5]: # cogemos el ultimo dag_run_id
last_run_id = dag_runs['dag_runs'][-1]['dag_run_id']
print(f"Consultando DAG Run: {last_run_id}\n")

# estado general
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}",
    auth=AUTH
)

run_detail = response.json()
print(f"Estado del DAG Run: {run_detail.get('state')}")
print(f"Inicio: {run_detail.get('start_date')}")
print(f"Fin: {run_detail.get('end_date')}")

# estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}/taskInstances",
    auth=AUTH
)

task_instances = response.json()
print(f"\nTareas ({len(task_instances.get('task_instances', []))}): \n")
for ti in task_instances.get('task_instances', []):

```

```
print(f" - {ti['task_id']}: {ti['state']} (intento {ti.get('try_number', '?')})")
```

Consultando DAG Run: manual__2026-03-17T11:48:41.951887+00:00

Estado del DAG Run: success

Inicio: 2026-03-17T11:48:42.382412+00:00

Fin: 2026-03-17T11:48:45.090153+00:00

Tareas (3):

- tarea0: success (intento 1)
- tarea2: success (intento 1)
- print_date: success (intento 1)

1.6 6. Consultar logs de una tarea

Tambien podemos ver los logs de cada tarea desde la API. Aqui vemos los de `print_date` (el `BashOperator` que imprime la fecha).

```
[6]: # logs de print_date
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{last_run_id}/taskInstances/print_date/logs/1",
    auth=AUTH,
    headers={"Accept": "text/plain"}
)

print("=== LOGS de print_date ===")
print(response.text[:2000])
```

=== LOGS de print_date ===

69aabdf8dd62

*** Found local files:

*** * /opt/airflow/logs/dag_id=test1/run_id=manual__2026-03-

17T11:48:41.951887+00:00/task_id=print_date/attempt=1.log

[2026-03-17T11:48:44.355+0000] {taskinstance.py:1979} INFO - Dependencies all met for dep_context=non-requeueable deps ti=<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00 [queued]>

[2026-03-17T11:48:44.366+0000] {taskinstance.py:1979} INFO - Dependencies all met for dep_context=requeueable deps ti=<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00 [queued]>

[2026-03-17T11:48:44.367+0000] {taskinstance.py:2193} INFO - Starting attempt 1 of 2

[2026-03-17T11:48:44.384+0000] {taskinstance.py:2217} INFO - Executing

<Task(BashOperator): print_date> on 2026-03-17 11:48:41.951887+00:00

[2026-03-17T11:48:44.391+0000] {standard_task_runner.py:60} INFO - Started process 672 to run task

```
[2026-03-17T11:48:44.397+0000] {standard_task_runner.py:87} INFO - Running:
['***', 'tasks', 'run', 'test1', 'print_date',
'manual__2026-03-17T11:48:41.951887+00:00', '--job-id', '47', '--raw', '--
subdir', 'DAGS_FOLDER/test.py', '--cfg-path', '/tmp/tmpspga81oj']
[2026-03-17T11:48:44.402+0000] {standard_task_runner.py:88} INFO - Job 47:
Subtask print_date
[2026-03-17T11:48:44.486+0000] {task_command.py:423} INFO - Running
<TaskInstance: test1.print_date manual__2026-03-17T11:48:41.951887+00:00
[running]> on host 69aabdf8dd62
[2026-03-17T11:48:44.593+0000] {taskinstance.py:2513} INFO - Exporting env vars:
AIRFLOW_CTX_DAG_EMAIL='test@test.com' AIRFLOW_CTX_DAG_OWNER='***'
AIRFLOW_CTX_DAG_ID='test1' AIRFLOW_CTX_TASK_ID='print_date'
AIRFLOW_CTX_EXECUTION_DATE='2026-03-17T11:48:41.951887+00:00'
AIRFLOW_CTX_TRY_NUMBER='1'
AIRFLOW_CTX_DAG_RUN_ID='manual__2026-03-17T11:48:41.951887+00:00'
[2026-03-17T11:48:44.595+0000] {subprocess.py:63} INFO - Tmp dir root location:
/tmp
[2026-03-17T11:48:44.597+0000] {subprocess.py:75} INFO - Running command:
['/usr/bin/bash', '-c', 'echo "La fecha es $(date)"]
[2026-03-
```

1.7 7. Lanzar el DAG con error controlado

En el código del video, si le pasamos `commit=1` la tarea0 lanza un `AirflowFailException` a propósito. Así se ve como funciona el error controlado y como las tareas que dependen de tarea0 quedan en `upstream_failed`.

```
[7]: # lanzar con commit=1 para que falle a proposito (como en el video)
response = requests.post(
    f"{BASE_URL}/dags/test1/dagRuns",
    json={"conf": {"commit": "1"}},
    auth=AUTH
)

error_run = response.json()
error_run_id = error_run.get('dag_run_id')
print(f"DAG Run ID (error): {error_run_id}")
print(f"Estado inicial: {error_run.get('state')}")

# esperamos
print("\nEsperando 30 segundos...")
time.sleep(30)

# comprobamos que ha fallado
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}",
    auth=AUTH
)
```

```

error_detail = response.json()
print(f"Estado final del DAG Run: {error_detail.get('state')}")

# vemos el estado de cada tarea
response = requests.get(
    f"{BASE_URL}/dags/test1/dagRuns/{error_run_id}/taskInstances",
    auth=AUTH
)

for ti in response.json().get('task_instances', []):
    print(f" - {ti['task_id']}: {ti['state']}")

```

DAG Run ID (error): manual__2026-03-17T11:49:13.628715+00:00
Estado inicial: queued

Esperando 30 segundos...
Estado final del DAG Run: failed

- tarea0: failed
- print_date: upstream_failed
- tarea2: upstream_failed

1.8 8. CLI de Airflow desde Jupyter

Como en el contenedor de Jupyter tambien esta instalado Airflow (lo puse en el Dockerfile para poder usar la CLI), podemos lanzar comandos directamente con !. Comparte la misma base de datos PostgreSQL que el scheduler asi que ve los mismos DAGs.

```

[8]: # listar DAGs (se ven los mismos que en la web de Airflow)
!airflow dags list 2>/dev/null

```

```

[2026-03-17T11:49:47.920+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.schemas
[2026-03-17T11:49:47.922+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.tables
[2026-03-17T11:49:47.923+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.types
[2026-03-17T11:49:47.923+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.constraints
[2026-03-17T11:49:47.924+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.defaults
[2026-03-17T11:49:47.924+0000] {plugins.py:37} INFO -
setup plugin alembic.autogenerate.comments
dag_id          | filepath          |
owner           | paused
=====+=====+=====+=====
dag_papermill   | dag_papermill.py  | airflow | False

```

```
test1          | test.py          | airflow | False
```

```
[9]: # tareas del DAG test1 (sacado del video de Albert Coronado)
!airflow tasks list test1 2>/dev/null
```

```
print_date
tarea0
tarea2
```

```
[10]: # version de airflow (la misma que usa el profesor en su entorno)
!airflow version 2>/dev/null
```

2.8.3

1.9 9. Ventajas de usar Jupyter como cliente de Airflow

- **Pruebas rapidas:** se pueden lanzar DAGs, cambiar parametros y relanzar sin salir del notebook
- **Automatizacion:** con la API REST se puede controlar Airflow por codigo, encadenando lanzamientos y consultas
- **Documentacion:** Jupyter mezcla codigo con texto, asi queda todo documentado paso a paso
- **API vs CLI:** la API funciona por red (entre contenedores), la CLI necesita tener Airflow instalado en la misma maquina. Desde Jupyter se pueden usar las dos

1.10 10. Papermill - ejecutar notebooks con parametros

Papermill permite ejecutar notebooks pasandoles parametros desde codigo. En el `notebook_parametrizable.ipynb` hay una celda con el tag `parameters` que tiene los valores por defecto. Cuando Papermill lo ejecuta, inyecta los valores nuevos encima de esos.

Primero lo probamos en local desde aqui y luego desde un DAG de Airflow.

```
[11]: import papermill as pm

# prueba local: ejecutar el notebook con parametros personalizados
pm.execute_notebook(
    input_path="notebook_parametrizable.ipynb",
    output_path="notebook_output_local.ipynb",
    parameters={
        "nombre": "Alumno",
        "repeticiones": 5
    }
)
print("Notebook ejecutado correctamente. Resultado en notebook_output_local.
↪ipynb")
```

```
Executing: 0%|          | 0/6 [00:00<?, ?cell/s]
```

```
Notebook ejecutado correctamente. Resultado en notebook_output_local.ipynb
```

1.11 11. Ejecutar el notebook desde un DAG de Airflow

El DAG `dag_papermill.py` usa un `PythonOperator` que llama a Papermill para ejecutar el notebook parametrizable. Los parametros se le pasan desde el `conf` del DAG Run, igual que hacíamos con el `test1`.

```
[12]: # activar el DAG de papermill
requests.patch(
    f"{BASE_URL}/dags/dag_papermill",
    json={"is_paused": False},
    auth=AUTH
)

# lanzarlo con parametros
response = requests.post(
    f"{BASE_URL}/dags/dag_papermill/dagRuns",
    json={"conf": {"nombre": "Airflow", "repeticiones": 4}},
    auth=AUTH
)

pm_run = response.json()
print(f"DAG lanzado - Run ID: {pm_run.get('dag_run_id')}")
print(f"Estado: {pm_run.get('state')}")
```

DAG lanzado - Run ID: manual__2026-03-17T11:49:59.096982+00:00
Estado: queued

```
[13]: # comprobar resultado
print("Esperando 30 segundos...")
time.sleep(30)

pm_run_id = pm_run.get('dag_run_id')
response = requests.get(
    f"{BASE_URL}/dags/dag_papermill/dagRuns/{pm_run_id}/taskInstances",
    auth=AUTH
)

for task in response.json().get("task_instances", []):
    print(f"  {task['task_id']}: {task['state']}")

print("\nEl notebook ejecutado se guarda en notebooks/notebook_output.ipynb")
```

Esperando 30 segundos...
ejecutar_notebook: success

El notebook ejecutado se guarda en `notebooks/notebook_output.ipynb`

1.12 12. Conclusiones

- Se ha montado un entorno aislado con Docker siguiendo la estructura del entorno del profesor (hadoop-lab_v6), con Apache Airflow 2.8.3 y Jupyter Notebook
- Desde Jupyter se ha usado Airflow de dos formas: como **cliente** (API REST y CLI) y como **contenido ejecutado** (Papermill)
- El DAG test1 del video de Albert Coronado demuestra PythonOperator, BashOperator, XCom, dependencias entre tareas y error controlado con AirflowFailException
- Con Papermill se pueden meter notebooks como tareas de Airflow sin tener que reescribirlos como scripts